Informatik I **Wintersemester 2025/26**
Dr. Henrik Brosenne Georg-August-Universität Göttingen
Paul Sarnighausen-Cahn Institut für Informatik

Übungsblatt 06

Allgemeine Information zum Bestehen der Prüfungsvorleistung.
Um die Prüfungsvorleistung zu bestehen, müssen Sie auf jedem Übungszettel mindestens 40% der Punkte in Theorie und Praxis erreichen. Es gibt jedoch zwei Ausnahmen: Sie dürfen auf zwei Zetteln bei den theoretischen Aufgaben und auf zwei Zetteln beim praktischen Teil unter die 40%-Grenze fallen, ohne dass dies zum Nichtbestehen führt.
Das bedeutet konkret: Wenn Sie beispielsweise auf einem Übungsblatt weniger als 40 % in der Theorie erreichen, können Sie den praktischen Teil trotzdem bestehen – und umgekehrt.

Theoretische Übungen

Abgabe bis Di., 06.01., 14 Uhr

Allgemein

Die Lösung dieser Aufgaben werden über das Lernmodul VIPS abgegeben.

Aufgabe 1 – 12 Punkte

Dictionaries und Hashing (i1-u-dh0)

Betrachten Sie die folgenden Aussagen zu Dictionaries in Python:

1. Jeder Schlüssel in einem Dictionary muss hashbar sein.
2. Zwei unterschiedliche Objekte können denselben Hashwert besitzen.
3. Gilt `a == b`, so gilt stets auch `hash(a) == hash(b)`.
4. Dictionaries speichern ihre Einträge nicht in sortierter Reihenfolge der Schlüssel.

- Markieren Sie alle korrekten Aussagen und begründen Sie jede Entscheidung in ein bis zwei Sätzen.
- Erklären Sie den Begriff Hash-Kollision.
- Erläutern Sie kurz, warum Hash-Kollisionen die Korrektheit eines Dictionaries nicht verletzen dürfen.

(12 Punkte)

Aufgabe 2 – 15 Punkte

Iteratoren und Seiteneffekte (i1-u-is0)

Betrachten Sie den folgenden Python-Code:

```
data = [1, 2, 3]
it = iter(data)
print(next(it))
data.append(4)
print(next(it))
```

Ziel des Codes: Der Code soll zwei aufeinanderfolgende Elemente der Datenstruktur `data` ausgeben.

- Beschreiben Sie allgemein, welche Probleme auftreten können, wenn eine Datenstruktur während der Iteration verändert wird.
- Erläutern Sie, warum Änderungen an der zugrunde liegenden Datenstruktur Auswirkungen auf bereits erzeugte Iteratoren haben können.
- Nennen Sie eine Möglichkeit, wie der Code so umgeschrieben werden kann, dass dieses Ziel erreicht wird, ohne dass während der Iteration Seiteneffekte auftreten.

(15 Punkte)

Aufgabe 3 – 10 Punkte

Sets und Mengenoperationen (i1-u-se0)

Gegeben seien die folgenden Python-Sets:

$$A = \{1, 2, 3, 5, 8\}, \quad B = \{2, 3, 4, 6, 8\}$$

- Geben Sie die Ergebnisse der folgenden Python-Operationen an und erläutern jeweils kurz deren Bedeutung:

```
A | B
A & B
A - B
A ^ B
```

- Erläutern Sie, warum Sets in Python keine definierte Iterationsreihenfolge garantieren, und beziehen Sie sich dabei kurz auf die zugrunde liegende Datenstruktur.
- Nennen Sie ein konkretes Anwendungsbeispiel aus der Informatik, bei dem Sets besser geeignet sind als Listen, und begründen Sie Ihre Wahl anhand der Eigenschaften von Sets.

(10 Punkte)

Aufgabe 4 – 13 Punkte

Iteratoren und Iterables (i1-u-it0)

Beantworten Sie die folgenden Fragen:

- Erläutern Sie den Unterschied zwischen einem Iterable und einem Iterator anhand der eingebauten Funktionen `iter` und `next`.
- Nennen Sie zwei eingebaute Python-Objekte, die iterierbar sind, und erläutern Sie kurz, wie man aus ihnen einen Iterator erzeugt.
- Erläutern Sie, warum ein Iterator typischerweise nur einmal vollständig durchlaufen werden kann.
- Python beendet das Durchlaufen eines Iterators automatisch, wenn keine weiteren Elemente vorhanden sind. Andere Programmiersprachen verwenden dafür häufig eine Methode `hasNext`. Diskutieren Sie mögliche Gründe, warum sich Python gegen eine solche Lösung entschieden hat.

(13 Punkte)
Hinweis:

- Überlegen Sie, was intern im Iterator-/Container-Objekt passiert.
- Python folgt dem Easier to Ask Forgiveness than Permission Prinzip.

Praktische Übung

Abgabe der Prüfsumme bis Di., 06.01., 14 Uhr
Testat ab Di., 06.01., 18 Uhr

Hilfe zum Bearbeiten der praktischen Übungen können Sie grundsätzlich jeden Tag in den Rechnerübungen bekommen.

- Bearbeiten Sie die praktischen Übungen in dem bereitgestellten Jupyter Notebook.
- Um die praktische Übung testieren zu lassen, **müssen** Sie einen Termin in der zugehörigen Rechnerübung reservieren. Ein Testat ohne Termin ist nicht möglich.
- Übermitteln Sie die Prüfsumme über Vips
- Öffnen Sie die Terminvergabe für diese praktische Übung und lassen Sie den von Ihnen reservierten Termin anzeigen.
- Zeigen Sie ihr Notebook und führen Sie alle zellen aus, damit am Ende die Prüfsumme angezeigt wird.
- Der CIP-Pool befindet sich hier: `https://lageplan.uni-goettingen.de/?ident=2412_1_1.UG_-1.110`

Aufgabe 1 – 15 Punkte

Einfache Turtle-Grafik: Quadratspirale (i1-p-qs0)

In dieser Aufgabe lernen Sie die grundlegende Verwendung des Moduls `jupyturtle` kennen. Ziel ist es, mit sehr einfachen Bewegungen eine Quadratspirale zu zeichnen.

Vorbereitung: Öffnen Sie eine Bash-Konsole in JupyterHub und führen Sie den folgenden Befehl aus:

```
pip install jupyturtle
```

Oder laden Sie das moduel wie folgt direkt im jupyter-notebook herunter:

```
from os.path import basename, exists

def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve

        local, _ = urlretrieve(url, filename)
        print("Downloaded " + str(local))
    return filename

download('https://github.com/ramalho/jupyturtle/releases/download/2024-03/
    jupyturtle.py');
```

Importieren Sie anschließend das Modul und erzeugen Sie ein Turtle-Objekt.

Schreiben Sie eine Funktion

```
def draw_square_spiral(t, steps, start_length, increment):
```

die eine Quadratspirale zeichnet.

- Die Spirale besteht aus `steps` Schritten.
- Im ersten Schritt bewegt sich die Turtle um `start_length` Einheiten vorwärts.
- Nach jedem Schritt dreht sich die Turtle um 90° nach links.
- Nach jedem Schritt wird die Schrittlänge um `increment` vergrößert.
- Verwenden Sie eine Schleife.
- Die Funktion soll das Turtle-Objekt `t` als Parameter erhalten und selbst kein Turtle-Objekt erzeugen.

Demonstrieren Sie die Verwendung der Funktion mit mindestens zwei verschiedenen Parameterwerten.
(15 Punkte)

Aufgabe 2 – 25 Punkte

Dictionary-basierte Textanalyse (i1-p-st4)

In dieser Aufgabe sollen Sie mithilfe von Dictionaries und Iteratoren eine einfache Textanalyse durchführen.

Gegeben sei eine Textdatei `woerter.txt`, die Wörter enthält, die durch Leerzeichen oder Zeilenumbrüche getrennt sind.

Verwenden Sie die folgende Code-Vorlage zum Einlesen der Datei:

```
def read_words(filename):
    with open(filename, encoding="utf-8") as f:
        for line in f:
            for word in line.split():
                yield word
```

Die Funktion liefert die Wörter der Datei nacheinander als Iterator.

- Verwenden Sie den gegebenen Iterator, um ein Dictionary der Form `wort` → `anzahl_vorkommen` zu erzeugen.
- Bestimmen Sie mithilfe geeigneter Iterationen die 10 häufigsten und die 10 seltensten Wörter.
- Berechnen Sie die durchschnittliche Wortlänge aller gelesenen Wörter.
- Erstellen Sie ein Dictionary, das Wörter nach ihrer Wortlänge gruppiert (Schlüssel: Wortlänge, Wert: Liste von Wörtern).
- Erstellen Sie ein Dictionary, das nur Wörter enthält, die ausschließlich aus alphabetischen Zeichen bestehen.

Hinweis:

- Behandlung von Wörtern mit Bindestrichen oder anderen Satzzeichen als ein Wort.

(25 Punkte)

Aufgabe 3 – 10 Punkte

ASCII-Weihnachtsbaum (i1-p-xm0)

In Jupyter Notebooks kann die Ausgabe von Text mit Leerzeichen und Sonderzeichen (z. B. Emojis) bei normaler Ausgabe mit `print` unschön dargestellt werden. Daher soll in dieser Aufgabe HTML zur formatierten Ausgabe verwendet werden.

Schreiben Sie eine Funktion

```
def christmas_tree(n, ornament="*"):
```

die einen Weihnachtsbaum erzeugt und als HTML-formatierten String zurückgibt.

Verwenden Sie dazu die folgende Vorlage zur Ausgabe:

```
from IPython.display import HTML

def display_tree(text):
    return HTML(f"<pre>{text}</pre>")
```

Der zurückgegebene Text soll an diese Funktion übergeben werden.

- `n` ist eine ungerade positive Zahl ≥ 3 und beschreibt die Breite der untersten Baumebene.
- Der Baum wird von oben nach unten aufgebaut: die oberste Ebene besitzt genau ein Schmuckzeichen, jede darunterliegende Ebene jeweils zwei mehr, bis die unterste Ebene `n` Schmuckzeichen besitzt.
- Jede Ebene ist horizontal zentriert auszugeben.
- Unter dem Baum wird ein Stamm ausgegeben, bestehend aus genau einer Zeile mit dem Zeichen `|`, ebenfalls zentriert.
- Falls `n` ungültig ist, soll die Funktion einen `ValueError` mit einer sinnvollen Fehlermeldung auslösen.

Hinweis: Die Funktion `christmas_tree` soll keine Ausgabe mit `print` erzeugen, sondern einen String zurückgeben, der anschließend mit `display_tree` dargestellt wird.

Optional (Bonus):

- Fügen Sie oberhalb des Baums einen Stern `+` hinzu.
- Legen Sie Geschenke unter den Baum mithilfe von Emojis.

(10 Punkte)